LAPORAN PRAKTIKUM

STRUKTUR DATA

“ QUEUE ARRAY ”

disusun Oleh:

FILZI JELILA INDA ROBBANI

(2511533019)

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas limpahan rahmat dan karunia-Nya sehingga praktikum Struktur Data kali ini dapat saya selesaikan dengan baik dan lancar. Shalawat serta salam semoga senantiasa tercurahkan kepada junjungan kita Nabi Muhammad SAW.

Laporan ini saya susun untuk memenuhi salah satu tugas pada mata kuliah Praktikum Struktur Data di Universitas Andalas. Laporan ini diharapkan dapat menambah wawasan mengenai penggunaan struktur data Queue pada ADT .

Saya menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum yang telah membimbing serta memberikan arahan selama proses pembelajaran di kelas maupun di laboratorium komputer. Selain itu, saya juga berterima kasih kepada teman-teman praktikan atas bantuan dan dukungan yang diberikan, sehingga laporan ini dapat terselesaikan dengan baik.

Saya menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, saya mohon maaf apabila terdapat kekurangan maupun kesalahan dalam penyusunan laporan ini. Harapan saya, semoga laporan ini dapat memberikan manfaat serta menambah pengetahuan bagi pembaca sekalian.

DAFTAR ISI

KATA PENGANTAR ..... **ii**
DAFTAR ISI ..... **iii**
BAB I PENDAHULUAN **4**
1.1 Latar Belakang 4
1.2 Tujuan 4
1.3 Manfaat 5
BAB II PEMBAHASAN **6**
2.1 Langkah Kerja Praktikum 6
2.1.1 Membuat kelas pertama (QueueArray) 6
2.1.2 Membuat kelas kedua (Mahasiswa) 8
2.1.3 Membuat program ketiga (MahasiswaDriver) 10
2.1.4 Membuat program keempat (DaftarKata) 12
2.1.5 Membuat program keempat (DaftarKataDriver) 13
2.2 Tugas Praktikum 15
2.2.1 Deskripsi Tugas 15
2.2.2 Langkah Tugas 15
BAB III PENUTUP **12**
3.1 Kesimpulan 12
DAFTAR PUSTAKA **13**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada praktikum sebelumnya, telah dipelajari mengenai Abstract Data Type (ADT) yang merupakan konsep dasar dalam pemrograman untuk merepresentasikan suatu struktur data beserta operasi-operasi yang dapat dilakukan terhadapnya. ADT membantu dalam memahami bagaimana data diorganisasikan dan dimanipulasi secara terstruktur tanpa bergantung pada implementasi tertentu.

Selanjutnya, pada praktikum kali ini dilakukan pengembangan dari konsep tersebut dengan mengimplementasikan struktur data menggunakan Queue dalam bahasa pemrograman Java. Queue adalah struktur data yang menyusun elemen-elemen data dalam urutan linier. Prinsip dasar dari struktur data ini adalah "First In, First Out" (FIFO) yang berarti elemen data yang pertama dimasukkan ke dalam antrean akan menjadi yang pertama pula untuk dikeluarkan.

Dalam praktikum ini, Queue digunakan dalam beberapa kasus, seperti pengolahan data sederhana, pengelolaan data dengan iterator, dan lain sebagainya. Hal ini bertujuan agar mahasiswa dapat memahami keterkaitan antara konsep ADT yang telah dipelajari sebelumnya dengan implementasi nyatanya dalam pemrograman.

Dengan demikian, praktikum ini menjadi lanjutan yang penting untuk memperkuat pemahaman mahasiswa dalam mengelola data secara dinamis, terstruktur, dan berbasis objek

1.2 Tujuan

Tujuan dilakukannya praktikum ini adalah sebagai berikut:

a. Memahami konsep dasar Queue (antrian) yang bekerja dengan prinsip FIFO dalam Java.

b. Memahami implementasi Queue menggunakan array secara manual, termasuk operasi dasarnya.
c. Memahami implementasi queue menggunakan LinkedList bawaan Java melalui interface Queue.
d. Memahami penggunaan queue dalam kasus tertentu seperti membalik urutan data (reverse) dengan bantuan struktur data stack.
e. Memahami cara melakukan iterasi atau traversal pada elemen queue menggunakan Iterator.
f. Melatih kemampuan dalam mengimplementasikan konsep teori struktur data ke dalam program Java.

1.3 Manfaat

a. Mahasiswa dapat memahami penggunaan struktur data dinamis dalam Java.
b. Mahasiswa dapat memahami bagaimana Queue bekerja dalam pengolahan data.
c. Meningkatkan kemampuan dalam mengelola data secara efisien dan fleksibel.
d. Mahasiswa dapat melatih kemampuan logika pemrograman dalam membuat operasi-operasi queue.
e. Mahasiswa dapat memahami hubungan queue dengan struktur data lain seperti stack.
f. Mahasiswa memiliki pengalaman praktis dalam debugging dan pengujian program queue.
g. Meningkatkan kemampuan problem solving dalam pemrograman.

BAB II
PEMBAHASAN

2.1 Langkah Kerja Praktikum

2.1.1 Membuat kelas pertama (QueueArray)

1) Pertama buka apl Eclipse IDE for Java Developers, lalu buatlah package baru dengan nama pekan4 terus pilih “New”, pilih class. Buat nama class “QueueArray_2511533019”.
2) Lalu, masukkan syntax seperti berikut.

```
1) package Pekan4_2511533019;
2)
3) public class QueueArray_2511533019 {
4)    int front_3019, rear_3019, size_3019;
5)    int capacity_3019;
6)    int array_3019[];
7)
8)    public QueueArray_2511533019 (int capacity) {
9)          this.capacity_3019 = capacity;
10)         front_3019 = this.size_3019 = 0;
11)         rear_3019 = capacity - 1;
12)         array_3019 = new int[this.capacity_3019];
13)   }
14)
15)   boolean isFull_3019(QueueArray_2511533019 queue) {
16)         return
   (queue.size_3019==queue.capacity_3019);
17)   }
18)
19)   boolean isEmpty_3019(QueueArray_2511533019 queue) {
20)         return (queue.size_3019 == 0);
21)   }
22)
23)   void enqueue(int item_3019) {
24)         if (isFull_3019(this))
25)               return;
26)         this.rear_3019 = (this.rear_3019 + 1)%
   this.capacity_3019;
27)         this.array_3019[this.rear_3019] = item_3019;
28)         this.size_3019 = this.size_3019 + 1;
29)         System.out.println(item_3019 + " enqueue to
   queue");
30)   }
31)
32)   int dequeue_3019() {
33)         if(isEmpty_3019(this))
34)               return Integer.MIN_VALUE;
35)         int item_3019 =
   this.array_3019[this.front_3019];
```

```
        this.front_3019 = (this.front_3019 + 1)% this.capacity_3019;
        this.size_3019 = this.size_3019-1;
        return item_3019;
    }
    int front_3019() {
        if (isEmpty_3019(this))
            return Integer.MIN_VALUE;
        return this.array_3019[this.front_3019];

    }
    int rear_3019() {
        if (isEmpty_3019(this))
            return Integer.MIN_VALUE;
        return this.array_3019[this.rear_3019];
    }
    //mencetak elemen antrian
    void display_3019(){
        int i_3019;
        if(front_3019 == rear_3019) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        //kunjungi dari belakang dan cetak
        for(i_3019 = front_3019; i_3019<rear_3019; i_3019++) {
            System.out.printf("%d <--", array_3019[i_3019]);
        }
        return;
        }
    }
```

Program pertama yaitu QueueArray_2511533019 merupakan implementasi struktur data queue menggunakan array secara manual. Pada class ini terdapat beberapa variabel seperti front_3019, rear_3019, size_3019, capacity_3019, dan array_3019[] yang digunakan untuk mengatur posisi elemen dalam antrian. Variabel front_3019 berfungsi menunjukkan elemen paling depan, rear_3019 menunjukkan elemen paling belakang, size_3019 menyimpan jumlah elemen saat ini, sedangkan capacity_3019 menentukan kapasitas maksimum queue. Konstruktor pada class ini digunakan untuk menginisialisasi kapasitas queue serta membuat array sesuai ukuran yang ditentukan. Program ini juga memiliki method isFull_3019() untuk mengecek apakah queue penuh dan isEmpty_3019() untuk mengecek apakah queue kosong.

Pada class ini juga terdapat method enqueue() yang berfungsi untuk menambahkan elemen baru ke dalam queue. Sebelum data dimasukkan, program akan memeriksa apakah queue sudah penuh. Jika belum penuh, maka posisi rear akan digeser menggunakan konsep circular queue dengan operasi modulo agar indeks dapat kembali ke awal array ketika mencapai batas akhir. Selain itu terdapat method dequeue_3019() yang berfungsi menghapus elemen paling depan sesuai prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Method front_3019() digunakan untuk melihat elemen terdepan tanpa menghapusnya, method rear_3019() digunakan untuk melihat elemen paling belakang, sedangkan method display_3019() digunakan untuk menampilkan seluruh isi queue.

2.1.2 Membuat kelas kedua (Mahasiswa)

1) Klik kanan package pekan2 dan pilih "New", pilih class. Lalu buat class baru dengan nama "QueueArrayDriver_2511533019."
2) Masukkan syntax seperti berikut.

```
1) package Pekan4_2511533019;
2)
3) public class QueueArrayDriver_2511533019 {
4)
5)    public static void main(String[] args) {
6)          // TODO Auto-generated method stub
7)          QueueArray_2511533019 queue_3019 = new
   QueueArray_2511533019(1000);
8)          queue_3019.enqueue(10);
9)          queue_3019.enqueue(20);
10)         queue_3019.enqueue(30);
11)         queue_3019.enqueue(40);
12)         System.out.println("Item di depan "+
   queue_3019.front_3019());
13)         System.out.println("Item paling belakang " +
   queue_3019.rear_3019());
14)         System.out.println("tampilan queue");
15)         queue_3019.display_3019();
16)         System.out.println();
17)         System.out.println(queue_3019.dequeue_3019()
   + " dihapus dari queue");
18)         System.out.println("Item di depan:" +
   queue_3019.front_3019());
```

```
19)         System.out.println("Item dibelakang: " +
    queue_3019.rear_3019());
20)         System.out.println("tampilan queue setelah
    satu data dihapus") ;
21)         queue_3019.display_3019();
22)    }
23)
24) }
```

Program kedua yaitu **QueueArrayDriver_2511533019** berfungsi sebagai class utama untuk menjalankan dan menguji class queue array yang telah dibuat sebelumnya. Pada program ini dibuat objek queue dengan kapasitas 1000 elemen. Setelah itu program melakukan proses enqueue dengan menambahkan data 10, 20, 30, dan 40 ke dalam queue. Selanjutnya program menampilkan elemen paling depan dan paling belakang menggunakan method front_3019() dan rear_3019(). Program juga menampilkan seluruh isi queue sebelum penghapusan data dilakukan. Setelah itu method dequeue_3019() dijalankan untuk menghapus elemen pertama, yaitu angka 10. Terakhir, program kembali menampilkan isi queue setelah satu elemen dihapus sehingga dapat terlihat perubahan data pada antrian.

3) Run program maka akan tampil output seperti berikut.

```
Console
<terminated> QueueArrayDriver_2511533019 [Java Application]
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
Item di depan 10
Item paling belakang 40
tampilan queue
10 <--20 <--30 <--
10 dihapus dari queue
Item di depan:20
Item dibelakang: 40
tampilan queue setelah satu data dihapus
20 <--30 <--
```

Saat program dirun sistem terlebih dahulu menambahkan data 10, 20, 30, dan 40 ke dalam queue menggunakan method enqueue(). Output yang muncul adalah pesan bahwa setiap data berhasil

dimasukkan ke antrian, seperti *"10 enqueue to queue"*, *"20 enqueue to queue"*, dan seterusnya. Setelah itu program menampilkan elemen paling depan yaitu 10 dan elemen paling belakang yaitu 40. Ketika method display_3019() dijalankan, program menampilkan isi queue sesuai urutan antrian yaitu 10, 20, 30, dan 40. Selanjutnya dilakukan proses dequeue_3019() sehingga data pertama yaitu 10 dihapus dari queue. Output kemudian menunjukkan bahwa elemen depan berubah menjadi 20 dan elemen belakang tetap 40. Saat queue ditampilkan kembali, isi antrian berubah menjadi 20, 30, dan 40. Hasil run ini membuktikan bahwa queue bekerja sesuai prinsip FIFO, yaitu data yang pertama masuk akan keluar lebih dahulu.

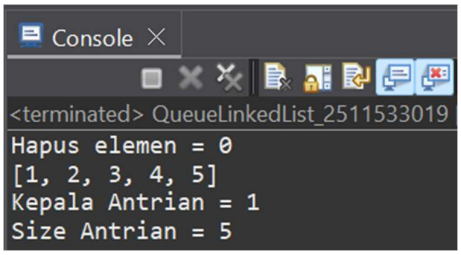
2.1.3 Membuat program ketiga (MahasiswaDriver)

1) Klik kanan package pekan4 dan pilih "New", pilih class. Lalu buat class baru dengan nama "QueueLinkedList_2511533019."
2) Lalu buat/masukkan syntax seperti berikut.

```
1) package Pekan4_2511533019;
2)
3) import java.util.LinkedList;
4) import java.util.Queue;
5)
6) public class QueueLinkedList_2511533019 {
7)
8)    public static void main(String[] args) {
9)          // TODO Auto-generated method stub
10)         Queue<Integer> q_3019 = new LinkedList<>();
11)         //tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
12)         for (int i = 0; i < 6; i++)
13)                q_3019.add(i);
14)         //menampilkan isi antrian.
15)         int hapus_3019 = q_3019.remove();
16)         System.out.println("Hapus elemen = " +
   hapus_3019);
17)         System.out.println(q_3019);
18)         //untuk melihat antrian terdpn
19)         int depan_3019= q_3019.peek();
20)         System.out.println("Kepala Antrian = "+
   depan_3019);
21)         int banyak_3019 = q_3019.size();
22)         System.out.println("Size Antrian = " +
   banyak_3019);
23)   }
24)
25)}
```

Program ketiga yaitu **QueueLinkedList_2511533019** merupakan implementasi queue menggunakan library bawaan Java, yaitu interface Queue dan class LinkedList. Pada program ini queue dibuat lebih sederhana karena tidak perlu mengatur indeks seperti pada array. Program menambahkan elemen angka 0 sampai 5 menggunakan perulangan for dan method add(). Setelah semua data dimasukkan, program menghapus elemen pertama menggunakan method remove() sehingga elemen paling depan keluar dari antrian. Selanjutnya program menggunakan method peek() untuk melihat elemen terdepan tanpa menghapusnya, kemudian method size() digunakan untuk mengetahui jumlah elemen yang tersisa dalam queue. Program ini menunjukkan bahwa penggunaan LinkedList lebih praktis dibandingkan implementasi manual menggunakan array.

3) Run program hingga keluar output seperti berikut, apabila tidak seperti pada gambar, cek kesalahan pada syntax/ kekurangan syntaxnya, dan perbaiki.



Setelah di run hasil run menunjukkan bahwa data angka 0 sampai 5 berhasil dimasukkan ke dalam queue menggunakan method add(). Setelah itu program menjalankan method remove() sehingga elemen pertama yaitu angka 0 dihapus dari antrian. Output menampilkan pesan *"Hapus elemen = 0"* dan isi queue berubah menjadi [1, 2, 3, 4, 5]. Selanjutnya method peek() menampilkan elemen paling depan yaitu angka 1 tanpa menghapusnya dari queue. Method size() juga menunjukkan jumlah elemen yang tersisa yaitu 5. Hasil run ini menunjukkan bahwa implementasi queue menggunakan LinkedList

berjalan dengan baik dan lebih simpel/mudah dibanding queue berbasis array.

2.1.4 Membuat program keempat (DaftarKata)

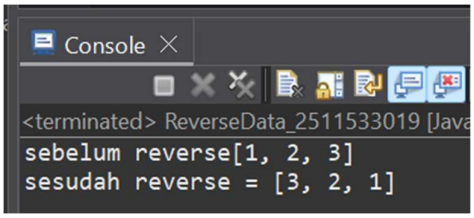
1) Klik kanan package pekan4 dan pilih “New”, pilih class. Lalu buat nama “ReverseData_2511533019”.
2) Masukkan syntax seperti berikut.

```
1) package Pekan4_2511533019;
2)
3) import java.util.LinkedList;
4) import java.util.Queue;
5) import java.util.Stack;
6)
7) public class ReverseData_2511533019 {
8)
9)  public static void main(String[] args) {
10)        // TODO Auto-generated method stub
11)        Queue<Integer> q_3019 = new
   LinkedList<Integer>();
12)        q_3019.add(1);
13)        q_3019.add(2);
14)        q_3019.add(3);// [1,2,3]
15)        System.out.println("sebelum reverse"+
   q_3019);
16)        Stack<Integer> s_3019 = new Stack<Integer>();
17)        while(!q_3019.isEmpty()) {//Q -> S
18)               s_3019.push(q_3019.remove());
19)        }
20)        while(!s_3019.isEmpty()) {//S -> Q
21)               q_3019.add(s_3019.pop());
22)        }
23)        System.out.println("sesudah reverse =
   "+q_3019);
24)
25) }
26)
27)}
```

Program keempat yaitu **ReverseData_2511533019** menunjukkan penerapan queue yang dikombinasikan dengan stack untuk membalik urutan data. Pada awal program dibuat queue yang berisi elemen 1, 2, dan 3. Data queue awal ditampilkan terlebih dahulu sebelum dilakukan proses reverse. Kemudian seluruh elemen queue dipindahkan ke dalam stack menggunakan perulangan while. Karena stack memiliki prinsip LIFO (Last In

First Out), urutan data di dalam stack menjadi terbalik. Setelah itu elemen dari stack dimasukkan kembali ke queue menggunakan perulangan kedua. Hasil akhirnya adalah urutan queue berubah dari [1,2,3] menjadi [3,2,1]. Program ini menunjukkan bagaimana dua struktur data yang berbeda dapat digunakan bersama untuk menyelesaikan suatu permasalahan.

3) Run program hingga mengeluarkan output seperti berikut.



Setelah di run nanti akan menampilkan isi queue sebelum dilakukan proses pembalikan data yaitu [1, 2, 3]. Setelah itu program memindahkan seluruh elemen queue ke dalam stack dan mengembalikannya lagi ke queue. Output akhir menunjukkan isi queue setelah proses reverse menjadi [3, 2, 1]. Hasil ini membuktikan bahwa kombinasi queue dan stack dapat digunakan untuk membalik urutan data.

2.1.5 Membuat program keempat (DaftarKataDriver)

1) Klik kanan package pekan4 dan pilih “New”, pilih class. Lalu buat nama “IterasiQueue_2511533019”.
2) Masukkan syntax seperti berikut.

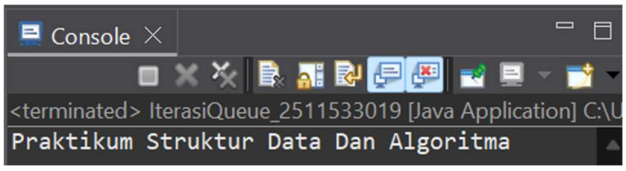
```
1) package Pekan4_2511533019;
2)
3) import java.util.Iterator;
4) import java.util.LinkedList;
5) import java.util.Queue;
6)
7) public class IterasiQueue_2511533019 {
8)
9)   public static void main(String[] args) {
10)        // TODO Auto-generated method stub
```

```
11)         Queue<String> q_3019= new LinkedList<>();
12)         q_3019.add ("Praktikum");
13)         q_3019.add ("Struktur");
14)         q_3019.add ("Data");
15)         q_3019.add ("Dan");
16)         q_3019.add ("Algoritma");
17)         Iterator<String> iterator_3019 =
   q_3019.iterator();
18)         while (iterator_3019.hasNext ()) {
19)                System.out.print (iterator_3019.next()
   + " ");
20)         }
21)   }
22)
23) }
```

Program terakhir yaitu **IterasiQueue_2511533019** digunakan untuk menampilkan isi queue menggunakan Iterator.

Pada program ini queue bertipe string dan diisi dengan beberapa kata yaitu "Praktikum", "Struktur", "Data", "Dan", dan "Algoritma". Setelah data dimasukkan ke dalam queue, program membuat objek iterator menggunakan method iterator(). Kemudian dilakukan perulangan while dengan kondisi hasNext() untuk memeriksa apakah masih terdapat elemen berikutnya. Jika masih ada, maka elemen akan ditampilkan menggunakan method next(). Output dari program ini adalah seluruh elemen queue ditampilkan secara berurutan. Program ini bertujuan untuk menunjukkan cara traversal atau penelusuran data pada queue menggunakan iterator.

3) Run program hingga keluar output seperti berikut, apabila error cek kembali syntax dan perbaiki kesalahan.



2.2 Tugas Praktikum

2.2.1 Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan struktur data Queue Antrian) menggunakan array dalam bahasa Java.

Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO First In First Out). Program harus mampu: Menambahkan pelanggan ke antrian (enqueue), menghapus pelanggan dari antrian (dequeue), menampilkan isi antrian dan mengecek kondisi antrian (kosong atau penuh)

2.2.2 Langkah Tugas

1) Klik kanan package pekan4 dan pilih "New", pilih class. Lalu buat nama "**AntrianLoket_2511533019**".
2) Lalu ketik atau buat syntax/kode program java seperti berikut.

```
1)  package Pekan4_2511533019;
2)
3)  import java.util.Scanner;
4)
5)  public class AntrianLoket_2511533019 {
6)          int front_3019, rear_3019, size_3019;
7)          int max_3019;
8)          String queue_3019[];
9)
10)         public AntrianLoket_2511533019 (int max_3019) {
11)                 this.max_3019 = max_3019;
12)                 front_3019 = this.size_3019 = 0;
13)                 rear_3019 = this.max_3019 - 1;
14)                 queue_3019 = new String[this.max_3019];
15)         }
16)
17)         boolean isFull_3019(AntrianLoket_2511533019 queue)
    {
18)                 return (queue.size_3019==queue.max_3019);
19)         }
20)
21)         boolean isEmpty_3019(AntrianLoket_2511533019 queue)
    {
22)                 return (queue.size_3019 == 0);
23)         }
24)
25)         void enqueue_3019(String data_3019) {
26)                 if (isFull_3019(this)) {
```

```
                        System.out.println("Antrian penuh!");
                        return;}
                rear_3019 = (rear_3019 + 1) % max_3019;
                queue_3019[rear_3019] = data_3019;
                size_3019++;
                System.out.println("Data berhasil ditambahkan ke antrian");
        }

        String dequeue_3019() {
                if(isEmpty_3019(this)) {
                        System.out.println("Antrian Kosong!");
                        return null;
                }
                String data_3019 = this.queue_3019[this.front_3019];
                this.front_3019 = (this.front_3019 + 1)% this.max_3019;
                this.size_3019 = this.size_3019-1;
                return data_3019;
        }
        String front_3019() {
                if (isEmpty_3019(this))
                        return null;
                return this.queue_3019[this.front_3019];

        }
        String rear_3019() {
                if (isEmpty_3019(this))
                        return null;
                return this.queue_3019[this.rear_3019];
        }

        void display_3019(){
                if(isEmpty_3019(this)) {
                        System.out.printf("\nAntrian Kosong\n");
                        return;
                }

                System.out.println("Isi Antrian : ");
                int i = this.front_3019;
                int no = 1;
                for(int count = 0; count < size_3019; count++) {
              System.out.println(no + ". " + queue_3019[i]);
              i = (i + 1) % max_3019;
              no++;
                }
        }
        void reverse_3019() {
                if (isEmpty_3019(this)) {
                        System.out.println ("Antrian kosong!");
                        return;
                }
                 for (int i = 0; i < size_3019 / 2; i++) {
                     int idx1_3019 = (front_3019 + i) % max_3019;
```

```java
                    int idx2_3019 = (front_3019 + size_3019 - 1 - i) % max_3019;

                    String temp = queue_3019[idx1_3019];
                    queue_3019[idx1_3019] = queue_3019[idx2_3019];
                    queue_3019[idx2_3019] = temp;
                }

        }
        public static void main(String[] args) {
                // TODO Auto-generated method stub
                Scanner input = new Scanner(System.in);
          AntrianLoket_2511533019 q = new AntrianLoket_2511533019(5);
          int pilih;
              System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
              System.out.println("1. Tambah Antrian");
              System.out.println("2. Hapus Antrian");
              System.out.println("3. Tampilkan Antrian");
              System.out.println("4. Reverse");
              System.out.println("5. Keluar");

              do {
                    System.out.print("Pilih menu: ");
                    pilih = input.nextInt();
                    input.nextLine();

                    switch (pilih) {
                        case 1:
                            System.out.print("Masukkan nama pelanggan: ");
                            String nama = input.nextLine();
                            q.enqueue_3019(nama);
                            break;

                        case 2:
                            String keluar = q.dequeue_3019();
                            if (keluar != null)
                                System.out.println(keluar + " telah dilayani");
                            break;

                        case 3:
                            q.display_3019();
                            break;

                        case 4:
                            q.reverse_3019();
                            q.display_3019();
                            break;

                        case 5:
                            System.out.println("Program selesai");
                            break;

                        default:
                            System.out.println("Pilihan tidak valid");
                    }
```

```
133)
134)             } while (pilih != 5);}
135)     }
```

Pada program ini dibuat beberapa variabel utama sesuai yang disampaikan pada deskripsi tugas, yaitu queue_3019[] sebagai tempat menyimpan data pelanggan, front_3019 sebagai penunjuk data paling depan, rear_3019 sebagai penunjuk data paling belakang, size_3019 untuk menghitung jumlah data dalam antrian, dan max_3019 sebagai kapasitas maksimum antrian.

Konstruktor digunakan untuk menginisialisasi ukuran array dan nilai awal variabel. Nilai rear_3019 diatur ke max_3019 - 1 agar mendukung sistem muter(*circular queue*) yaitu antrian yang bisa berputar kembali ke awal array jika sudah mencapai batas akhir.

Method isFull_3019() digunakan untuk mengecek apakah antrian sudah penuh, sedangkan isEmpty_3019() digunakan untuk mengecek apakah antrian kosong.

Method enqueue_3019() berfungsi untuk menambahkan data ke dalam antrian. Jika antrian penuh, maka akan muncul pesan. Jika tidak, data akan dimasukkan ke posisi belakang dan nilai rear_3019 akan digeser menggunakan operasi modulo (%) agar tetap sesuai dengan konsep circular.

Method dequeue_3019() digunakan untuk menghapus data dari antrian bagian depan. Jika antrian kosong, maka akan muncul pesan. Jika tidak, data diambil dari posisi front_3019, lalu posisi front_3019 digeser ke depan dan jumlah data dikurangi.

Method display_3019() digunakan untuk menampilkan isi antrian dari depan ke belakang sebanyak jumlah data yang ada.

Method reverse_3019() digunakan untuk membalik urutan antrian. Prosesnya dilakukan dengan menukar data dari depan dan belakang secara bertahap sampai setengah jumlah data.

Pada bagian main, program menggunakan menu interaktif agar user bisa memilih operasi seperti menambah antrian, menghapus antrian, menampilkan data, membalik antrian, atau keluar dari program. Perulangan do-while digunakan supaya program terus berjalan sampai user memilih keluar.

3) Run program, dan inputkan sesuai perintah. Contoh output dengan berbagai input yang telah saya masukkan :

```
Console
<terminated> AntrianLoket_2511533019 [Java Application] C:\Users\Filzi Jelila

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: filzi
Data berhasil ditambahkan ke antrian
Pilih menu: 1
Masukkan nama pelanggan: faliska
Data berhasil ditambahkan ke antrian
Pilih menu: 3
Isi Antrian :
1. filzi
2. faliska
Pilih menu: 4
Isi Antrian :
1. faliska
2. filzi
Pilih menu: 2
faliska telah dilayani
Pilih menu: 3
Isi Antrian :
1. filzi
Pilih menu: 5
Program selesai
```

BAB III
PENUTUP

3.1 Kesimpulan

Dari seluruh praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data queue (antrian) merupakan struktur data yang bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Praktikum ini berhasil menunjukkan dua cara implementasi queue, yaitu menggunakan array secara manual dan menggunakan LinkedList dari library Java, di mana masing-masing memiliki kelebihan tersendiri. Implementasi menggunakan array memberikan pemahaman lebih dalam terkait pengelolaan indeks seperti front dan rear serta konsep *circular queue*, sedangkan penggunaan LinkedList lebih sederhana dan praktis karena memanfaatkan fitur bawaan Java.

Selain itu, praktikum ini juga menunjukkan bahwa queue dapat dikombinasikan dengan struktur data lain seperti stack untuk menyelesaikan permasalahan tertentu, contohnya dalam proses reverse data. Mahasiswa juga memahami cara melakukan iterasi atau traversal pada queue menggunakan Iterator. Pada studi kasus antrian loket, konsep queue dapat diterapkan secara nyata dalam kehidupan sehari-hari, seperti sistem pelayanan pelanggan, sehingga mempermudah pemahaman terhadap fungsi dan kegunaan queue.

Secara keseluruhan, praktikum ini membantu meningkatkan pemahaman konsep dasar struktur data queue, melatih kemampuan implementasi dalam bahasa pemrograman Java, serta mengembangkan logika berpikir dalam menyelesaikan masalah berbasis antrian.

DAFTAR PUSTAKA

[1] Wahyudi, "*Queuet"* PowerPoint slides, Praktikum Struktur Data, Universitas Andalas, Padang, 2025.

[2] Oracle, "Queue Interface (Java Platform SE Documentation)," [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html